T.R.

GEBZE TECHNICAL UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

BIOMETRIC-BASED ENCRYPTED VAULT IMPLEMENTATION

ÖMER FARUK OLKAY

SUPERVISOR
PROF. DR. ERKAN ZERGEROĞLU

GEBZE
2026

T.R.
GEBZE TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

BIOMETRIC-BASED ENCRYPTED VAULT IMPLEMENTATION

ÖMER FARUK OLKAY

SUPERVISOR
PROF. DR. ERKAN ZERGEROĞLU

2026
GEBZE

GEBZE TECHNICAL UNIVERSITY	GRADUATION PROJECT JURY APPROVAL FORM

This study has been accepted as an Undergraduate Graduation Project in the Department of Computer Engineering on 18/06/2026 by the following jury.

JURY

Member

(Supervisor) : Prof. Dr. Erkan Zergeroğlu

Member : Dr. Alp Arslan Bayrakçlı

ABSTRACT

Encrypted storage is an effective way to protect sensitive files. However, password-based access can be difficult to manage in daily use and may reduce usability. This project presents a biometric based encrypted digital vault that combines fingerprint-controlled access with standard Linux disk encryption. The system allows the user to unlock a protected storage area with fingerprint verification and then use it like a normal mounted filesystem. The project covers the main vault lifecycle, including setup, biometric authorization, lock and unlock operations, virtual storage support, emergency locking, and deletion.

The hardware side of the prototype uses an AS608 fingerprint sensor, an STM32 Blue Pill microcontroller, and a PL2303 USB-UART bridge. Fingerprint templates are stored inside the sensor, so the host computer does not store raw biometric data. Access to the vault key is controlled by the firmware side. The host application uses the key only for the required vault operation and clears it from memory after use. The storage layer is based on Linux Unified Key Setup (LUKS2) and dm-crypt, which are standard Linux mechanisms for encrypted storage volumes. The encrypted volume uses AES-256-XTS encryption, and an ext4 filesystem is created inside the unlocked vault.

The software architecture is divided into a Qt/QML graphical interface, a host-side controller, a serial communication layer, and a vault manager. This separation makes the system easier to understand. It also separates user interaction, firmware communication, and storage operations. Special attention is given to asynchronous workflows and state consistency, because vault operations can be interrupted by wrong fingerprints, device removal, sensor disconnection, busy mount points, or process termination.

The system was evaluated through functional workflow tests, unit-level software tests, serial communication tests, filesystem usage tests, stress tests, and failure case tests. The results show that the prototype performs the intended vault workflows correctly and handles common failure cases in a controlled way. The project demonstrates how biometric authorization, embedded firmware control, and standard Linux storage encryption can be combined into a usable secure storage workflow.

Keywords: biometric authentication, encrypted storage, digital vault, LUKS2, dm-crypt, fingerprint sensor, STM32.

ÖZET

Şifreli depolama, hassas dosyaların korunmasında etkili bir yöntemdir. Ancak parola tabanlı erişim günlük kullanımda ek yönetim yükü oluşturabilir ve kullanım kolaylığını azaltabilir. Bu projede, parmak izi kontrollü erişim ile standart Linux disk şifreleme mekanizmalarını birleştiren biyometrik tabanlı şifreli bir dijital kasa sistemi geliştirilmiştir. Sistem, kullanıcının korumalı bir depolama alanını parmak izi doğrulaması ile açmasını ve kasa açıldıktan sonra bu alanı normal bir dosya sistemi gibi kullanmasını sağlar. Proje; kurulum, biyometrik yetkilendirme, kilitleme ve kilit açma işlemleri, sanal depolama desteği, acil kilitleme ve kasa silme gibi temel kasa yaşam döngüsü adımlarını kapsamaktadır.

Prototipin donanım tarafında AS608 parmak izi sensörü, STM32 Blue Pill mikrodenetleyici ve PL2303 USB-UART dönüştürücü kullanılmaktadır. Parmak izi şablonları sensör içerisinde tutulduğu için ana bilgisayarda ham biyometrik veri saklanmaz. Kasa anahtarına erişim mikrodenetleyici içerisindeki gömülü yazılım tarafından kontrol edilir. Ana bilgisayar bu anahtarı yalnızca gerekli kasa işlemi sırasında kullanır ve işlem sonrasında bellekten temizler. Depolama katmanı, şifreli depolama birimleri için kullanılan standart Linux mekanizmaları olan LUKS2 ve dm-crypt üzerine kuruludur. Şifreli birimde AES-256-XTS şifreleme kullanılır ve açılmış kasa içerisinde ext4 dosya sistemi oluşturulur.

Yazılım tarafı Qt/QML tabanlı kullanıcı arayüzü, ana bilgisayarda çalışan kontrol katmanı, seri haberleşme katmanı ve kasa yöneticisi olarak ayrılmıştır. Bu ayrım kullanıcı etkileşimini, gömülü yazılım haberleşmesini ve depolama işlemlerini daha anlaşılır hale getirir. Kasa işlemleri yanlış parmak izi, cihazın çıkarılması, sensör bağlantısının kopması, meşgul bağlama noktaları veya süreç kesintileri gibi durumlarla karşılaşabileceği için asenkron iş akışları ve durum tutarlılığına özellikle dikkat edilmiştir.

Sistem; işlevsel akış testleri, yazılım birim testleri, seri haberleşme testleri, dosya sistemi kullanım testleri, stres testleri ve hata durumu testleri ile değerlendirilmiştir. Testler, prototipin hedeflenen kasa iş akışlarını doğru şekilde gerçekleştirdiğini ve yaygın hata durumlarını kontrollü biçimde ele alabildiğini göstermiştir. Proje, biyometrik doğrulama, gömülü yazılım kontrolü ve standart Linux şifreli depolama altyapısının kullanılabilir bir güvenli depolama iş akışı içinde birleştirilebileceğini göstermektedir.

Anahtar Kelimeler: biyometrik kimlik doğrulama, şifreli depolama, dijital kasa, LUKS2, dm-crypt, parmak izi sensörü, STM32.

ACKNOWLEDGEMENT

I would like to thank my project supervisor Prof. Dr. Erkan Zergeroğlu for the guidance and feedback provided during the development of this project. I would also like to thank my friends who provided valuable feedback.

Ömer Faruk Olkay

LIST OF SYMBOLS AND ABBREVIATIONS

Symbol or Abbreviation	: **Explanation**
AES	: Advanced Encryption Standard
AS608	: Optical fingerprint sensor module used in the project
CBC	: Cipher Block Chaining
CLI	: Command-Line Interface
CTR	: Counter Mode
dm-crypt	: Linux kernel device-mapper encryption target
ECB	: Electronic Codebook
ext4	: Fourth extended filesystem used inside the unlocked vault
FAT32	: File Allocation Table 32-bit filesystem
FUSE	: Filesystem in Userspace
GUI	: Graphical User Interface
HCI	: Human-Computer Interaction
IV	: Initialization Vector
LUKS	: Linux Unified Key Setup
LUKS2	: Version 2 of Linux Unified Key Setup
NTFS	: New Technology File System
PC	: Personal Computer
QML	: Qt Modeling Language
Qt	: Cross-platform application development framework
RAM	: Random Access Memory
STM32	: ARM Cortex-M based microcontroller family used in the project
TPM	: Trusted Platform Module
UART	: Universal Asynchronous Receiver/Transmitter
USB	: Universal Serial Bus
XTS	: XEX-based Tweaked-codebook mode with ciphertext stealing

CONTENTS

Abstract iv

Özet v

Acknowledgement vi

List of Symbols and Abbreviations vii

Contents ix

List of Figures x

List of Tables xi

1 Introduction 1

2 Background and Literature Review 3

2.1 Symmetric Encryption for Stored Data 3

2.2 XTS Mode for Disk Encryption 4

2.3 LUKS2 and dm-crypt Based Storage Encryption 5

2.4 Biometric Authentication and Key Access 6

2.5 Hardware-Backed Key Management 7

2.6 Embedded Hardware Used for Biometric Access 7

2.7 Existing Secure Storage Approaches 8

2.8 FUSE-Based Filesystem Approach 10

2.9 Summary 10

3 System Design and Architecture 11

3.1 Design Goals 11

3.2 Overall Architecture 13

3.3 Hardware Architecture 14

3.4 Software Architecture 14

3.5 User Interface Design Considerations 15

3.6 Vault Storage Design 16

3.7 Authentication and Key Access Flow 17

3.8 Vault Lifecycle 20

3.8.1 Vault Creation 20
3.8.2 Formatting and Initialization 21
3.8.3 Unlock 21
3.8.4 Lock and Force-Lock 21
3.8.5 Auto-Lock and Emergency Lock 22
3.8.6 Deletion 22
3.9 Asynchronous Workflow and State Consistency 25
3.10 Design Alternatives 26
3.11 Security Considerations 28

4 Implementation **30**
4.1 Implementation Overview 30
4.2 Physical Prototype 31
4.3 Host Application 31
4.4 Firmware Communication 32
4.5 Vault Manager 32
4.6 Runtime Recovery 33

5 Testing and Evaluation **34**
5.1 Testing Approach 34
5.2 Functional Test Cases 35
5.3 Failure and Edge Case Tests 37
5.4 Filesystem Usage and Stress Tests 38
5.5 Serial Communication Tests 39
5.6 Evaluation Summary 39

6 Conclusion and Future Work **41**
6.1 Project Outcomes 41
6.2 Limitations 42
6.3 Future Work 42
6.4 Final Remarks 43

Bibliography **45**

Appendices **46**

LIST OF FIGURES

3.1 Use-case diagram of the digital vault system. 12
3.2 System architecture of the digital vault. 13
3.3 Authentication and key-access flow. 19
3.4 Unlock sequence diagram. 20
3.5 Vault lifecycle state diagram. 24

4.1 Physical prototype including the AS608 fingerprint sensor, STM32 Blue Pill, and PL2303 USB-UART bridge. 31

LIST OF TABLES

2.1 Comparison of common encryption modes for storage usage. 4
2.2 Storage layers used in the project. 6
2.3 Comparison of related secure storage approaches. 9

3.1 Vault storage layout options. 17
3.2 Summary of vault lifecycle operations. 23
3.3 Comparison of considered storage design alternatives. 27
3.4 Threat and mitigation summary. 29

4.1 Main implementation units. 30

5.1 Test strategy overview. 35
5.2 Functional test cases. 36
5.3 Failure and edge case test cases. 37
5.4 Filesystem usage and stress test cases. 38
5.5 Serial communication test cases. 39

6.1 UART packet format summary. 46
6.2 Host-to-firmware command summary. 47
6.3 Firmware event and response summary. 47

1. INTRODUCTION

The project, titled *Biometric-Based Encrypted Vault Implementation*, presents a small secure storage system that combines biometric authentication with encrypted storage management. The main goal is to create a vault that behaves like a normal mounted storage area after it is unlocked. Access to the vault is controlled by a fingerprint sensor and an embedded firmware component. The project uses standard Linux storage encryption mechanisms and focuses on the surrounding system, including enrollment, unlock, lock, formatting, device management, and communication between the host computer and the biometric hardware.

The hardware side of the project is based on an AS608 fingerprint sensor, an STM32 Blue Pill microcontroller, and a PL2303 USB-UART bridge. Fingerprint templates are kept inside the fingerprint sensor, so the host computer does not store raw biometric data. The cryptographic key is managed by the firmware side and becomes available only after a successful fingerprint verification. The key is not permanently stored on the host computer. It is used for the vault operation and then cleared from memory. The transfer between the firmware and the host uses a protected channel, so the key is not exposed as plaintext during communication.

On the storage side, the project uses a Linux Unified Key Setup (LUKS2)/dm-crypt based design with AES-256-XTS encryption. A vault can be created on a physical block device or inside a virtual container backed by a sparse image and loop device. During setup, the system formats the encrypted vault partition, prepares the filesystem, and registers the vault information used by the graphical interface. During normal use, a successful biometric verification unlocks the encrypted storage and mounts it as an accessible filesystem. The user can then work with files through the operating system as usual. When the vault is locked, the mount is removed and the encrypted mapper is closed.

The graphical interface provides the main workflows of the system: creating a vault, formatting storage, enrolling authorized fingerprints during vault creation, unlocking and locking the vault, viewing vault details, and monitoring security-related events. The project also includes safety-oriented behavior such as automatic locking after inactivity, emergency locking when the sensor connection is lost or when the vault's block device is removed, and a force-lock option for cases where files inside the vault are still being used by another process.

The rest of this report explains the project in more detail. Chapter 2 introduces the background technologies used in the system. Chapter 3 describes the system architecture and design decisions. Chapter 4 presents the implementation of the hard-

ware interface, vault manager, and graphical application. Chapter 5 discusses testing and evaluation. Chapter 6 presents the conclusion, limitations, and possible future improvements.

2. BACKGROUND AND LITERATURE REVIEW

This chapter reviews the main technical ideas behind the project. The purpose is not to present every detail of each technology, but to explain the storage encryption, biometric authentication, and secure storage approaches that shaped the design of the digital vault.

2.1. Symmetric Encryption for Stored Data

Encrypted storage systems generally rely on symmetric-key cryptography. In this model, the same secret key is used for both encryption and decryption. This is practical for disk encryption because large amounts of data must be processed quickly while the user reads and writes files.

Several symmetric algorithms were reviewed before deciding on the storage encryption approach. AES is a standardized 128-bit block cipher with 128-, 192-, and 256-bit key options [1]. ChaCha20 is a fast stream cipher and is attractive on systems without dedicated AES hardware, but its common standardized use is with nonce-based authenticated encryption protocols rather than as the direct block cipher for sector-based disk encryption [2]. Serpent is a conservative block cipher design with a large security margin, but this project did not need to trade the selected storage stack for that extra margin [3].

Twofish is a flexible 128-bit block cipher with key sizes up to 256 bits and several performance trade-offs [4]. Camellia has the same basic block and key-size interface as AES and was designed for both software and hardware implementations [5]. Both are valid algorithms to study, but the project storage layer already follows the LUKS2/dm-crypt AES-XTS path described in the next section. For that reason, AES is the more direct fit for this implementation.

For this project, AES-256 was selected because it best matches the actual storage design. The choice was not only about cryptographic strength. AES is standardized, widely studied, and supported by a large software and security community. It is also directly supported by common Linux storage tools, including the LUKS2/dm-crypt stack used in this project. This matters in practice because the project can use existing, tested tooling instead of adding a custom cryptographic layer. The following section explains why AES-XTS is a suitable mode for encrypted block devices, which makes AES the most natural choice for this implementation.

2.2. XTS Mode for Disk Encryption

A block cipher alone is not enough for encrypting a disk. Storage encryption needs a mode of operation that handles sectors or blocks independently, supports random access, and avoids exposing obvious patterns when similar data appears in different disk locations.

XTS-AES is a mode designed specifically for confidentiality on storage devices. NIST Special Publication 800-38E approves XTS-AES for protecting data on storage devices, while also noting an important limitation: XTS-AES does not provide authentication of the data or its source [6]. This distinction matters. XTS protects confidentiality, but it does not by itself detect every possible malicious modification. In this project, the primary goal is to prevent unauthorized reading of vault contents when the storage device is lost, removed, or accessed without biometric authorization.

The project uses AES-256-XTS through the LUKS2/dm-crypt layer. This choice fits the project better than modes such as ECB, CBC, or CTR for the reasons summarized in Table 2.1. In short, it is a better match for disk encryption and random access.

Table 2.1: Comparison of common encryption modes for storage usage.

Mode	Advantages	Limitations	Suitability for vault storage
ECB	Simple and parallelizable	Reveals repeated plaintext patterns	Not suitable
CBC	Historically common and simple to understand	Less convenient for random-access disk sectors; needs careful IV handling	Not preferred
CTR	Fast and random-access friendly	Reusing counters or nonces is dangerous; integrity is not provided	Possible, but requires careful design
XTS	Designed for sector-based storage; hides repeated data better across positions	Provides confidentiality but not authentication	Suitable for disk encryption

2.3. LUKS2 and dm-crypt Based Storage Encryption

Linux Unified Key Setup 2 (LUKS2) is used in this project as the container format for encrypted storage, while dm-crypt performs the block-level encryption in the Linux kernel [7], [8]. LUKS2 stores the metadata needed to identify and unlock the encrypted volume. dm-crypt then handles the actual encryption and decryption of block device sectors after the vault is opened.

This approach keeps the project away from custom cryptographic storage design. Instead of implementing encryption and filesystem semantics manually, the project uses a standard encrypted block device and then creates a normal filesystem inside it. In the current implementation, the vault filesystem is formatted as ext4 after the LUKS2 container is opened [9]. After mounting, applications can read and write files normally. When the vault is locked, the filesystem is unmounted and the dm-crypt mapper is closed.

Using LUKS2/dm-crypt also makes the system easier to reason about. The project code is responsible for the workflow around the vault: fingerprint enrollment, biometric unlock, key handling, mount management, auto-lock, emergency lock, and user interface behavior. The lower-level encryption of disk sectors remains inside a mature operating system storage layer [8].

Table 2.2: Storage layers used in the project.

Layer	Role	Project usage
Physical block device or loop device	Provides the raw storage area	USB storage, disk partition, or sparse-image-backed virtual container
LUKS2 container	Stores encrypted volume metadata and unlock structure	Defines the protected vault volume
dm-crypt mapper	Performs sector encryption and decryption after unlock	Exposes the decrypted block device while the vault is open
ext4 filesystem	Stores user files inside the decrypted volume	Mounted for normal file access after biometric unlock
Vault manager and GUI	Controls setup, unlock, lock, logging, and device workflows	Provides the user-facing secure vault experience

2.4. Biometric Authentication and Key Access

Biometric authentication is used in this project as an access-control mechanism, not as an encryption algorithm. The fingerprint sensor checks whether the presented finger belongs to an authorized user. After successful verification, the firmware side can allow the vault key to be used for unlocking encrypted storage.

This separation is important. Fingerprint data should not replace cryptographic keys directly. Biometric measurements can be noisy, and they cannot be changed like passwords if they are compromised. A safer design keeps fingerprint templates inside the sensor and uses the biometric match only as a gate before releasing or using a cryptographic key. In this project, fingerprint templates are stored on the AS608 sensor, while the host computer works only with verification results and short-lived key material.

The access model is also intentionally simple: authorized fingerprints are registered when the vault is created, and the access list is not changed later during normal use. This reduces the risk of accidental or unauthorized changes to vault ownership after deployment.

2.5. Hardware-Backed Key Management

A common idea in secure storage systems is to keep long-term encryption keys away from the general-purpose host operating system. This does not mean the host never handles secret material, but it reduces how much permanent trust is placed in host-side files and application storage.

Industrial systems use this idea in different ways. BitLocker can use a Trusted Platform Module (TPM) to help protect key release and verify the boot environment before a protected volume is unlocked [10]. Hardware encrypted USB drives, such as IronKey-class devices, move encryption and access control into the device itself. The host sees an unlocked storage device only after device-side authentication succeeds [11]. Self-encrypting drives follow a similar direction by performing encryption inside the storage device controller and exposing plaintext storage only after successful authorization.

The digital vault follows the same principle at a project scale. It is not the same as a TPM-backed system or a commercial encrypted USB drive, but it uses a similar idea: long-term key control is moved away from ordinary host-side files. In this project, the firmware manages access to the main vault key, and fingerprint verification acts as the gate for using that key. The host computer does not permanently store the key. During an unlock operation, any key material used by the host is short-lived and is cleared after the storage layer has been opened. The communication channel between firmware and host is protected so that the key is not transferred as plaintext.

2.6. Embedded Hardware Used for Biometric Access

The hardware side of the project is intentionally simple. The AS608 fingerprint sensor handles fingerprint enrollment, template storage, and matching. Because the templates never leave the sensor, the host computer never sees raw biometric data. The STM32 Blue Pill acts as the firmware controller between the sensor and the host-side software. It coordinates biometric operations and controls the key-access logic. The PL2303 USB-UART bridge provides the serial communication path between the STM32 firmware and the computer.

This hardware split gives each component a clear role. The sensor focuses on biometric matching, the microcontroller controls the security decision and firmware protocol, and the host software manages encrypted storage and the graphical workflow.

2.7. Existing Secure Storage Approaches

Several existing systems solve similar problems with different trade-offs. BitLocker is a full-volume encryption feature in Windows. Microsoft describes it as a protection mechanism for entire volumes, mainly addressing data theft from lost, stolen, or improperly decommissioned devices. BitLocker can also work with a Trusted Platform Module (TPM), which helps protect startup integrity and key release [10]. This approach is strong for operating-system drives and managed Windows environments, but it is closely tied to the Windows platform.

Commercial encrypted USB drives, such as Kingston IronKey products, move more of the security boundary into the external device. These products commonly use hardware encryption, password or keypad-based unlocking, brute-force protection, and sometimes tamper-resistant designs. Public analyses of IronKey-class devices show why this design is attractive: encryption and access control can be handled inside the device rather than depending entirely on the host operating system [11]. At the same time, hardware-based storage products are not automatically perfect. Research on secure USB devices has also shown that firmware and hardware manipulation are realistic threat areas [12].

The project is closer to a hybrid approach. Like BitLocker and LUKS-based systems, it uses operating-system storage encryption instead of implementing a new filesystem encryption format. Like hardware encrypted drives, it moves authentication and key control toward external hardware. The difference is that the project combines a fingerprint sensor, STM32 firmware, and a Linux vault manager rather than using a closed commercial encrypted drive.

Table 2.3: Comparison of related secure storage approaches.

Approach	Main idea	Main trade-off
BitLocker	Full-volume encryption integrated with Windows; can use TPM, PIN, password, or recovery key depending on configuration	Strong platform integration, but Windows-specific
LUKS2/dm-crypt	Linux encrypted block device approach with a normal filesystem inside the decrypted mapper	Strong Linux integration, but authentication and key policy must be designed around it
Hardware encrypted USB drive	Encryption and access control are handled inside the storage device hardware	Portable and self-contained, but often closed and costly
Self-encrypting drive	Storage controller performs encryption internally and unlocks after authorization	Efficient and hardware-contained, but depends heavily on vendor implementation
Application-level encrypted container	Files are protected by an application-managed container and opened through that application	Portable, but every file access depends on application behavior
FUSE-based encrypted filesystem	User-space filesystem code intercepts file operations and can apply custom encryption logic	Flexible, but filesystem behavior and performance must be handled carefully
This project	Fingerprint-controlled key access is combined with standard Linux encrypted storage	Keeps storage encryption standard while adding biometric hardware control

2.8. FUSE-Based Filesystem Approach

Filesystem in Userspace (FUSE) allows a filesystem to run as a normal user-space program. The Linux kernel sends file requests to this program through callbacks [13], [14].

This idea is useful for a vault because the project code can be placed directly in the file access path. For example, a read operation could decrypt data, and a write operation could encrypt data. If the key is not available, the system could ask for fingerprint verification before continuing.

This level of control was part of the initial design direction. However, it also adds filesystem complexity. This trade-off is discussed together with the final storage decision in Chapter 3.

2.9. Summary

The literature and existing systems point to a practical design direction: use well-established encryption primitives and storage layers, and focus the project work on authentication, key handling, device management, and user workflow. AES-256-XTS provides a suitable confidentiality layer for disk encryption. Biometric authentication is useful as a key-release gate, but it should not be treated as a cryptographic key by itself. Existing systems such as BitLocker and hardware encrypted USB drives show that secure storage usually depends on a combination of encryption, key protection, recovery policy, and usability. The proposed digital vault follows the same general idea while adapting it to a biometric embedded-hardware setting.

3. SYSTEM DESIGN AND ARCHITECTURE

This chapter describes the design of the proposed digital vault system. The previous chapter introduced the background technologies; this chapter explains how those technologies are combined into a working architecture.

3.1. Design Goals

The main design goal of the project is to provide an encrypted storage vault that can be opened through biometric authorization while still behaving like normal storage after it is unlocked. The system should not require the user to manually run cryptographic commands or remember a separate disk password during daily use. At the same time, the encryption itself should rely on standard and mature Linux storage mechanisms instead of custom cryptographic code.

The design goals can be summarized as follows:

- protect vault contents with standard block-level encryption;
- use fingerprint verification as the main access-control step;
- avoid storing raw biometric data on the host computer;
- keep long-term key control on the firmware side;
- use short-lived key material on the host only when required for vault operations;
- support common vault lifecycle actions such as create, format, unlock, lock, force-lock, and delete;
- prevent the application from remaining in an inconsistent state after failed, interrupted, or asynchronous operations;
- provide a graphical interface for the major workflows without hiding important security events from the user.

These goals shaped the decision to separate the project into hardware, host-control, and storage-management components. Each part has a limited responsibility, which makes the system easier to test and easier to reason about.

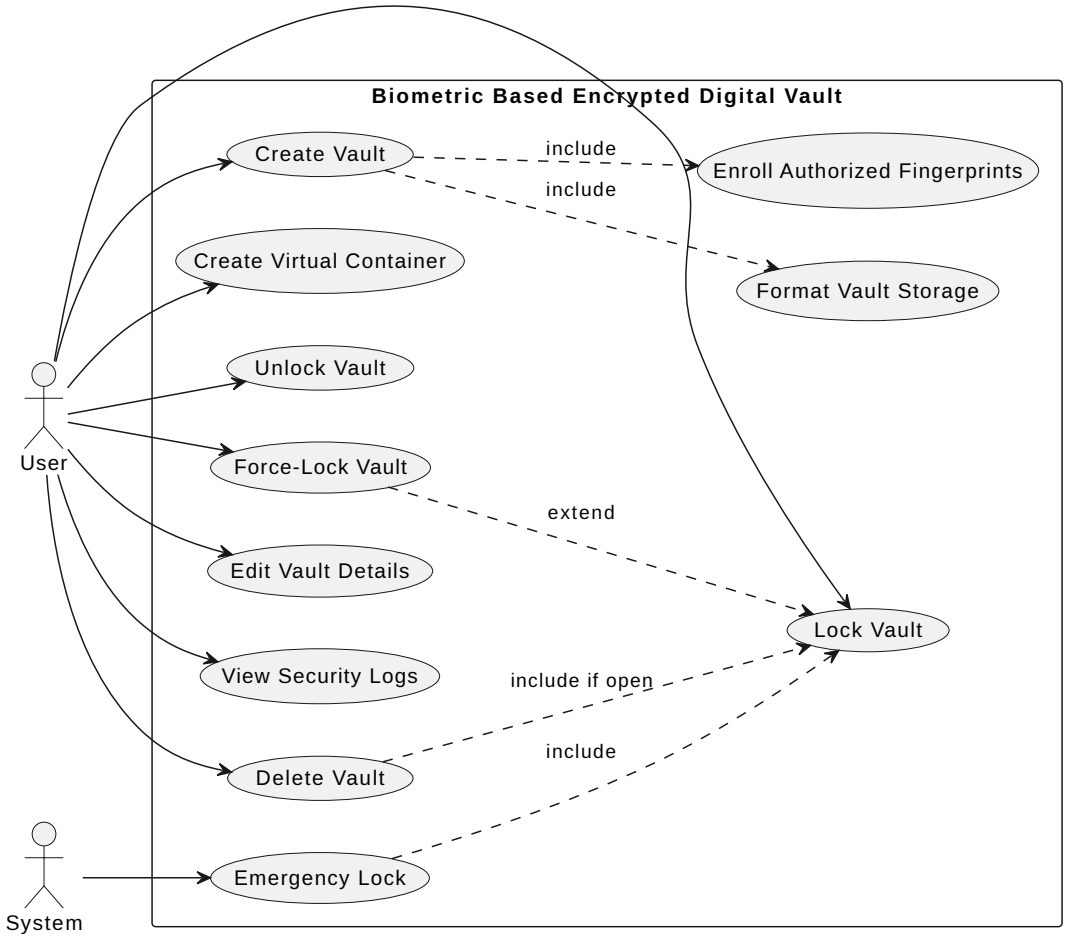


Figure 3.1: Use-case diagram of the digital vault system.

3.2. Overall Architecture

The system is built around three main layers: the biometric hardware layer, the host application layer, and the encrypted storage layer. The biometric hardware layer consists of the AS608 fingerprint sensor, the STM32 Blue Pill microcontroller, and the PL2303 USB-UART bridge. The host application layer contains the Qt/QML graphical interface, the serial/HAL communication logic, and the vault controller. The encrypted storage layer uses LUKS2/dm-crypt and an ext4 filesystem inside the unlocked vault.

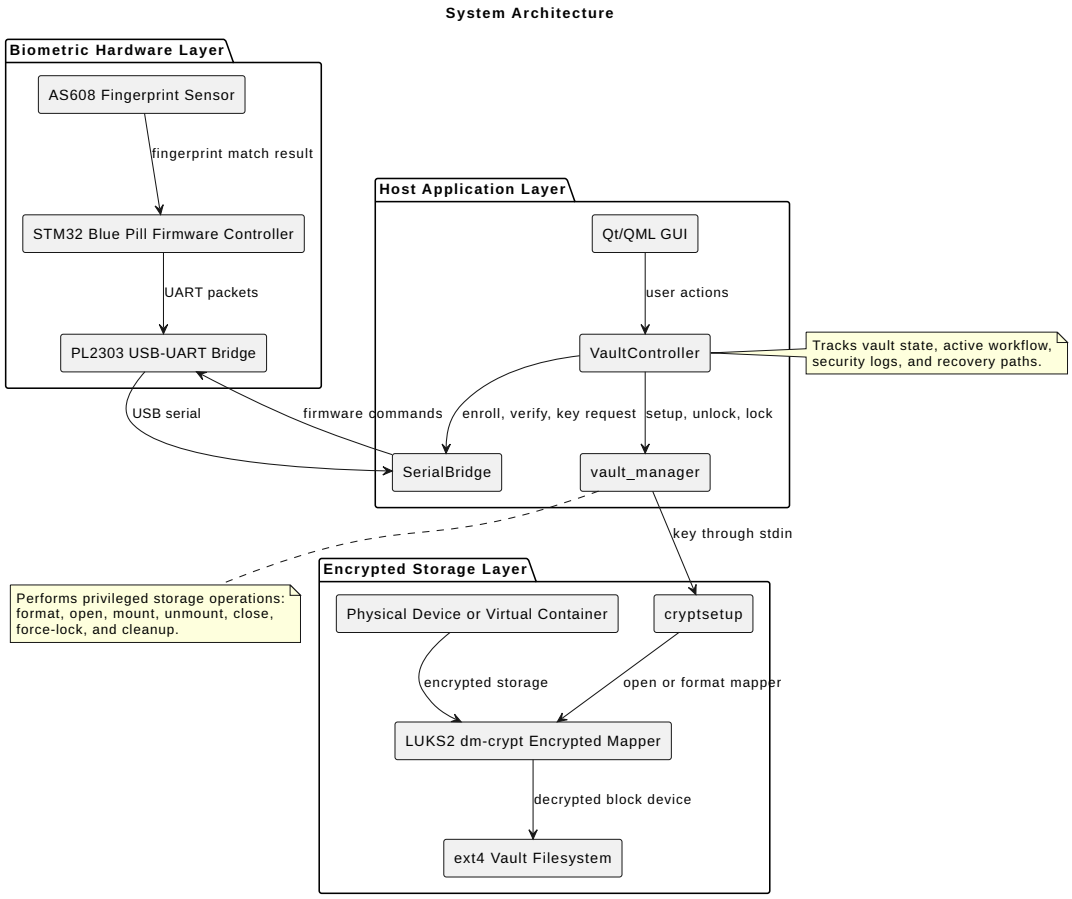


Figure 3.2: System architecture of the digital vault.

The host application does not directly implement disk encryption. Instead, it coordinates the user workflow and calls the vault manager for system-level storage operations. The vault manager performs operations such as LUKS formatting, opening the encrypted mapper, mounting the filesystem, unmounting the vault, and closing the mapper. This keeps storage operations in a smaller command-line component while the GUI remains responsible for interaction, status display, and workflow control.

The firmware side is responsible for biometric access decisions and key-release

control. The STM32 firmware communicates with the fingerprint sensor and the host, while the host application coordinates the resulting storage workflow. This keeps the overall architecture separated into biometric authorization, host-side workflow control, and encrypted storage management.

3.3. Hardware Architecture

The hardware architecture consists of three main components: the AS608 fingerprint sensor, the STM32 Blue Pill microcontroller, and the PL2303 USB-UART bridge. The AS608 sensor is responsible for biometric enrollment and matching. Fingerprint templates are stored inside the sensor, so the host computer does not need to store raw fingerprint data or template data.

The STM32 Blue Pill works as the controller between the fingerprint sensor and the host computer. It receives commands from the host, drives the fingerprint operation flow, checks sensor results, and sends structured responses back to the host. The firmware also represents the key-control boundary of the system. The vault key is generated and controlled on the firmware side, and key access is allowed only after a valid fingerprint verification.

The PL2303 USB-UART bridge provides the physical communication path between the STM32 firmware and the computer. From the host side, this appears as a serial port. This choice keeps the hardware connection simple and easy to test while still allowing the project to define its own command and response protocol above the serial link.

The firmware protocol is intentionally small. The host can request operations such as enroll, verify, delete, list enrolled fingerprints, cancel, check status, and request key material after verification. The firmware responds with success, failure, timeout, busy, sensor error, or key-data messages. This small command set is enough for the vault workflows while keeping the firmware interface understandable.

3.4. Software Architecture

The software architecture is divided into the graphical application, the serial communication layer, the vault controller, and the vault manager. This separation was chosen because not every part of the application should have the same responsibility. The vault controller manages the GUI-side workflow, while the vault manager handles storage requests coming from the controller and returns the operation result back to the application.

The Qt/QML graphical interface provides the user-facing workflows. It presents

available storage devices, vault status, setup steps, fingerprint prompts, security logs, and vault details. The GUI does not directly format disks or call cryptographic tools. Instead, it calls methods exposed by the vault controller and updates itself based on controller state.

The vault controller is the central coordinator on the GUI side of the host application. It tracks the selected device, current lock state, setup state, fingerprint operation state, auto-lock timer, registered vaults, and security logs. It starts long-running operations asynchronously and updates the GUI through signals and state properties. It also decides which workflow is active, such as vault setup, unlock, delete confirmation, or fingerprint enrollment. The controller does not directly mount disks or format encrypted volumes; it validates the current state and requests the required operation from the vault manager.

The serial communication layer, represented by the serial bridge, handles the STM32 connection. It opens the configured serial port, sends firmware commands, parses firmware responses, emits sensor events, and reports completed operations back to the vault controller. This keeps low-level packet handling away from the GUI and allows the vault controller to work with higher-level events such as "verification completed" or "key received".

The vault manager is a separate command-line component on the lower-level storage side. It handles system-level storage operations, including LUKS formatting, opening and closing the dm-crypt mapper, mounting and unmounting the ext4 filesystem, creating loop devices, deleting vault partitions, and force-closing busy vault mounts. The GUI does not run these commands directly. For example, during unlock, the controller first completes the fingerprint and key-access workflow, then asks the vault manager to open and mount the encrypted storage. The manager performs the operation and returns success or failure so that the GUI can show the correct state.

3.5. User Interface Design Considerations

The graphical interface was designed with basic human-computer interaction principles in mind. Since the project includes destructive actions such as formatting a storage device or deleting a vault, the interface should help the user understand the current state and avoid accidental actions.

One important principle is recognition over recall. The user should not need to remember device paths, command-line options, mount states, or cryptographic commands. Instead, the interface presents discovered storage devices, registered vaults, lock status, sensor status, and available actions as visible choices. This reduces the mental effort required to operate the vault.

The interface also tries to preserve the user's existing mental model of storage. A user is already familiar with choosing a storage device, opening a folder, copying files, and closing or removing the device when finished. The vault workflow follows this real-world model: after biometric unlock, the vault appears as an accessible folder; after locking, it becomes unavailable again. The encryption and mapper operations remain behind this familiar interaction pattern.

The interface also follows the idea of visibility of system status. States such as locked, unlocked, setup running, sensor disconnected, fingerprint prompt active, or mount busy are shown through the GUI instead of being hidden in terminal output. This helps the user's mental model match the real system state, especially because vault operations can be asynchronous and may take time.

Error prevention was also considered in the workflow design. Formatting, force-formatting, vault deletion, and force-locking are not treated as ordinary one-click actions. The interface uses warnings, previews, confirmation dialogs, and biometric gates where appropriate. These steps are not only usability details; they also reduce the chance of data loss or unintended security changes.

3.6. Vault Storage Design

The vault storage design is based on an encrypted block device. The project can work with a physical block device, such as a USB storage device, or with a virtual container created from a sparse image and attached as a loop device. This gives the user two storage choices: a vault can be created on real removable storage, or a virtual storage file can be created and used as the vault device.

During vault setup, the selected storage target is checked before formatting. If the target already contains a LUKS vault, the setup operation is stopped and the user is informed. If existing data is detected, formatting is also stopped unless the user explicitly chooses a force-format path. Vault creation overwrites the target, so this check is the last line of defense against wiping the wrong device.

The storage target can be used as a full encrypted vault or split into two partitions. In the split mode, the first partition becomes the encrypted vault partition and the second partition becomes a normal storage partition. The normal partition can be formatted with a selected filesystem such as exFAT, FAT32, ext4, NTFS, XFS, or Btrfs. This design allows a device to contain both a protected vault area and a normal visible storage area.

The encrypted vault itself is formatted as a LUKS2 container using AES-256-XTS through dm-crypt. After the container is opened, an ext4 filesystem is created inside the decrypted mapper. The setup flow temporarily mounts the new filesystem

to set file ownership for the real user, then unmounts it and closes the mapper. After setup, the vault remains locked until the user performs a biometric unlock.

Table 3.1: Vault storage layout options.

Mode	Storage layout	Use case
Full-device vault	The selected block device or loop device is formatted as one LUKS2 vault	Simple secure storage device with one protected area
Split device	Partition 1 is the encrypted vault; partition 2 is normal storage	Removable storage with both private and ordinary space
Virtual container	A sparse image is attached as a loop device and then used like a block device	User-created virtual storage that can also be used as a vault

When the vault is unlocked, the vault manager opens the LUKS2 container as a dm-crypt mapper and mounts the ext4 filesystem at the configured mount point. From the user's perspective, the vault then behaves like a normal folder. When the vault is locked, the filesystem is unmounted and the mapper is closed. This separation keeps encryption below the filesystem layer while still allowing applications and file managers to interact with files normally after unlock.

3.7. Authentication and Key Access Flow

The authentication flow is designed around the idea that fingerprint verification controls access to the vault key. The fingerprint itself is not used as a cryptographic key. Instead, the AS608 sensor performs biometric matching, the STM32 firmware controls the verification flow, and the host requests key access only after successful verification.

There are two main key-access situations: vault setup and vault unlock. During setup, the system needs the vault key to format the new LUKS2 container. During unlock, it needs the same key to open the existing LUKS2 container. In both cases, the key is managed on the firmware side and used by the host only for the minimum time needed to perform the storage operation.

The host-side controller treats fingerprint operations as explicit flows. A verification result is interpreted according to the current state: setup confirmation, unlock,

delete confirmation, or another controlled operation. The same physical action, scanning a finger, can mean very different things depending on the active workflow, so the controller has to know which flow it is in before acting on a result.

After key material reaches the host for a setup or unlock operation, it is passed to the vault manager for the required `cryptsetup` command through standard input instead of command-line arguments. The key buffer is wiped after use, and no persistent host-side copy is kept.

Key access is only needed for operations that open or initialize encrypted storage. Vault setup uses the key to format the new LUKS2 container and create the ext4 filesystem. Vault unlock uses the same key to open the existing container and mount the filesystem. Locking does not need the key, because it only unmounts the filesystem and closes the mapper. Vault deletion may still use a biometric gate as a safety step, but the vault key is not needed to remove encrypted data or container metadata.

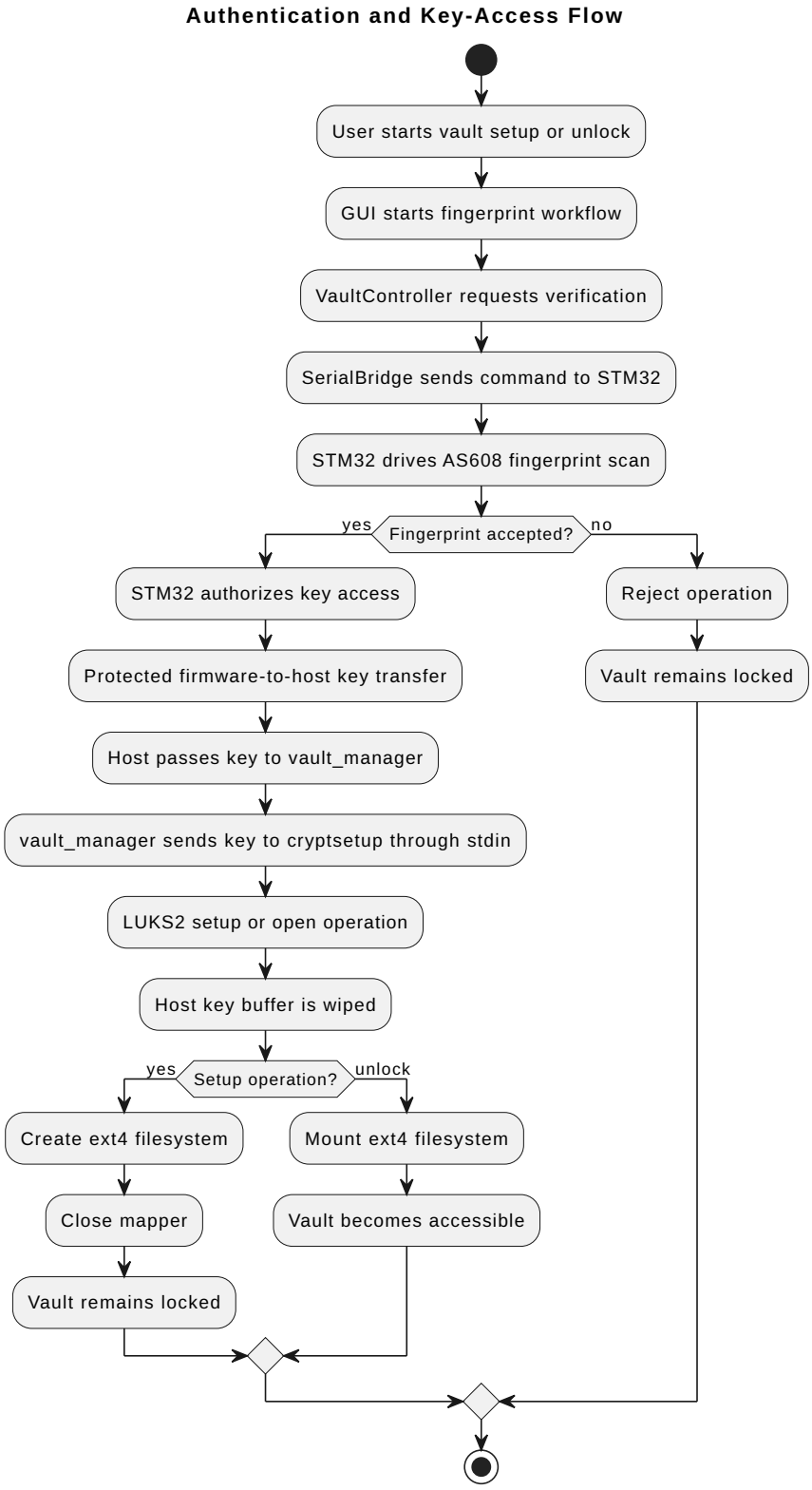


Figure 3.3: Authentication and key-access flow.

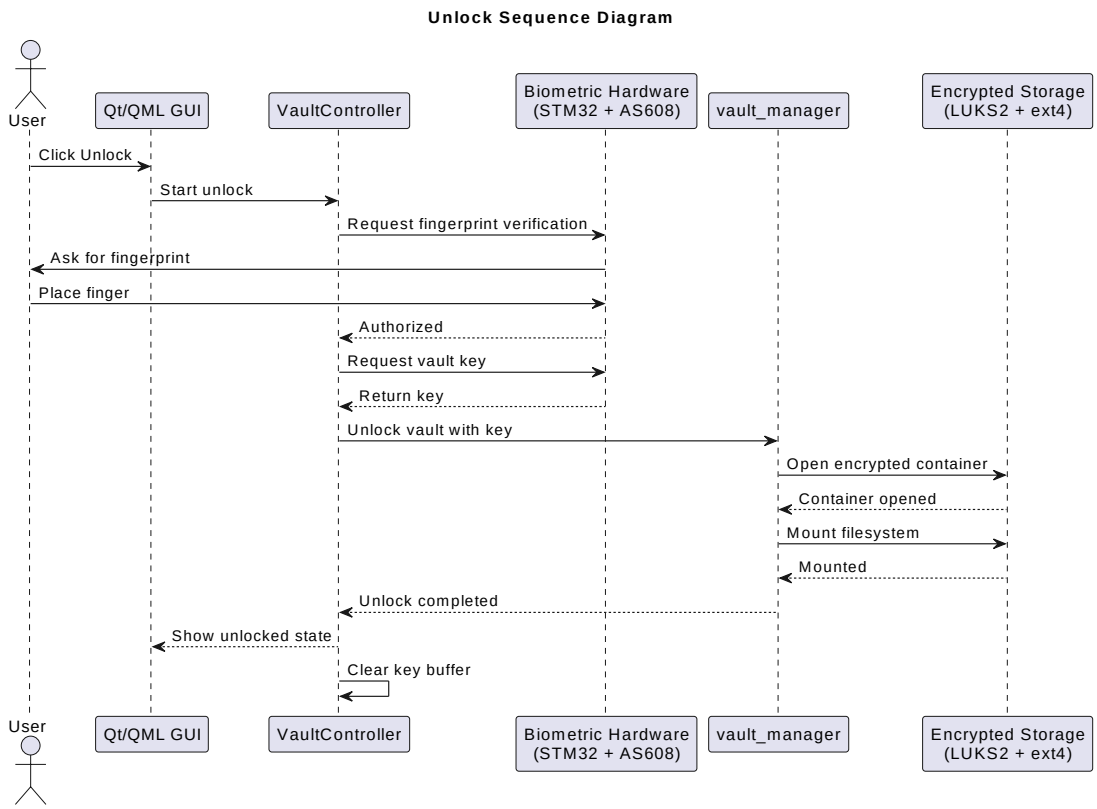


Figure 3.4: Unlock sequence diagram.

3.8. Vault Lifecycle

The vault lifecycle describes how a storage target becomes a protected vault, how it is opened for use, and how it is safely closed or removed. The system is designed so that each lifecycle step has a clear beginning, expected result, and recovery behavior.

3.8.1. Vault Creation

Vault creation starts with selecting a storage target. The target can be a physical block device or a virtual container created by the application. For a virtual container, the application creates a sparse image file, attaches it as a loop device, records the backing image in the vault registry, and then allows the user to continue with the normal vault setup flow.

During setup, the user provides the vault name, storage size options, normal partition options if a split layout is used, and authorized fingerprint information. In the access policy, the fingerprints that can open the vault are defined during vault creation and are not modified later during normal use.

3.8.2. Formatting and Initialization

Formatting is the destructive part of the lifecycle, so the selected device is checked before it is overwritten. If an existing LUKS container or existing data is detected, the setup flow stops unless the user intentionally chooses the force-format path.

After the checks, the vault manager prepares the storage layout. If split mode is selected, it creates a vault partition and a normal partition. The normal partition is formatted with the selected ordinary filesystem. The vault partition is formatted as a LUKS2 container with AES-256-XTS, opened as a dm-crypt mapper, and then formatted as ext4. Finally, the new filesystem is briefly mounted for ownership setup, unmounted, and closed. At the end of initialization, the vault is registered in the application registry and remains locked.

3.8.3. Unlock

Unlocking starts from the graphical interface. The user selects a vault and begins the unlock operation. The controller starts a fingerprint verification flow through the serial bridge and firmware. If the fingerprint is accepted, key access is requested from the firmware side.

The vault manager then uses the key material to open the LUKS2 container and mount the ext4 filesystem. After the vault is mounted, the key material used by the host is wiped. The GUI updates the state to show that the vault is accessible, starts the auto-lock timer, and can open the mount point in the file manager.

3.8.4. Lock and Force-Lock

Normal locking unmounts the vault filesystem and closes the dm-crypt mapper. If no process is using the mount point, this operation is straightforward and the vault returns to the locked state.

If the mount point is busy, the system does not silently ignore the problem. It reports that the vault is in use and can offer a force-lock path. Force-lock first verifies that the mount point belongs to the vault, then stops processes holding the mount, unmounts the filesystem, and closes the mapper. This behavior is useful when a file manager, terminal, or application keeps a file open inside the vault.

3.8.5. Auto-Lock and Emergency Lock

The application supports automatic locking after inactivity. When the vault is open and the configured timeout is reached, the controller starts the lock operation. This reduces the chance of leaving the vault mounted for a long time after the user stops interacting with it.

Emergency locking is used when an unexpected event could leave the system in an unsafe or inconsistent state. Examples include sensor disconnection, application shutdown, startup recovery, or removal of the block device used by the vault. In these cases, the controller tries to reconcile the runtime state and close any remaining mapper or mount that belongs to the vault.

3.8.6. Deletion

Vault deletion is also treated as a controlled lifecycle step. If the vault is open, the system first tries to lock it. For a virtual container, deletion detaches the loop device and removes the backing sparse image. For a physical vault partition, deletion wipes the encrypted vault signature or metadata from the selected partition. The registry is then updated so that the deleted vault is no longer shown as an available managed vault.

Table 3.2: Summary of vault lifecycle operations.

Operation	Main system actions	Expected final state
Create virtual storage	Create sparse image, attach loop device, register backing file	Virtual storage is available for vault setup
Setup vault	Check device, partition if needed, LUKS2 format, ext4 creation, registry update	Vault exists and is locked
Unlock vault	Verify fingerprint, access key, open LUKS2 mapper, mount ext4 filesystem	Vault is mounted and accessible
Lock vault	Unmount filesystem and close mapper	Vault is locked
Force-lock vault	Stop holders if needed, unmount, close mapper, reconcile runtime state	Vault is locked even after busy mount state
Auto-lock	Trigger normal lock after inactivity timeout	Vault returns to locked state
Emergency lock	Reconcile mapper and mount after disconnection or abnormal state	Runtime state is cleaned up
Delete vault	Lock if needed, wipe vault or remove backing image, update registry	Vault is removed from managed devices

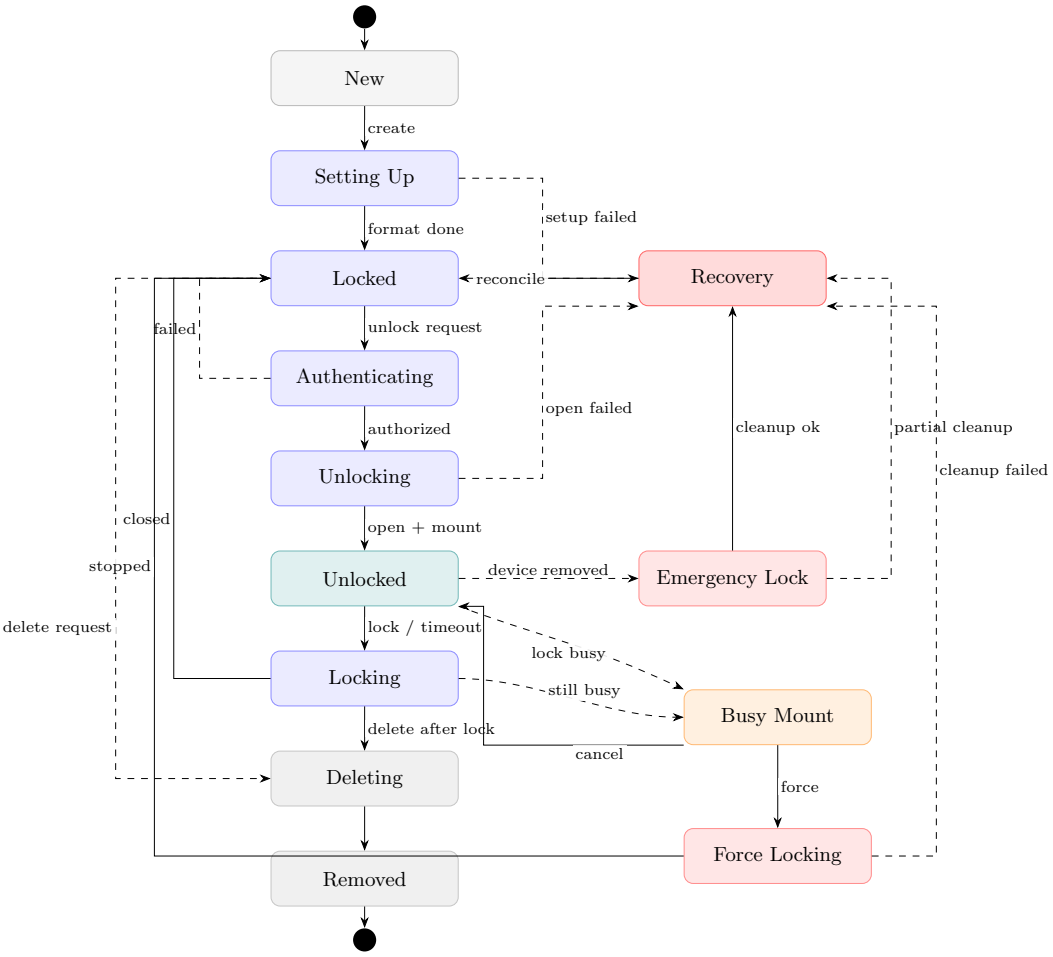


Figure 3.5: Vault lifecycle state diagram.

3.9. Asynchronous Workflow and State Consistency

One of the important design challenges in this project is keeping the system out of unstable intermediate states. Vault operations are not simple instant function calls. Formatting a device, opening a LUKS container, mounting a filesystem, waiting for fingerprint input, detecting sensor disconnection, and handling user cancellation are all operations that can take time or fail halfway through. If these operations are handled carelessly, the system can be left in a confusing state, such as a mapper being open without a mount point, a GUI showing the wrong lock status, or a fingerprint enrollment flow continuing after the sensor connection is lost.

For this reason, the host-side design uses asynchronous operations and explicit state tracking. Long-running vault operations are executed outside the main GUI flow so that the interface does not freeze. The controller tracks whether setup, unlock, lock, or fingerprint operations are active, and the UI reacts to these states instead of assuming that every operation succeeds immediately.

The design also treats cleanup as part of the normal workflow, not only as error handling. When a lock operation fails because the mount point is busy, the system reports this state and can offer a force-lock path. When the application starts, it can reconcile leftover runtime state such as an open mapper or stale mount. When the sensor or the block device is disconnected, emergency locking is used to reduce the chance of leaving the vault accessible in an unintended way.

State-machine style thinking is especially important for fingerprint operations. Enrollment, verification, unlock, setup, force-lock confirmation, and delete confirmation flows have different meanings even though they all use the same physical sensor. Treating them as separate states prevents a callback from one flow from accidentally completing another flow. For example, a successful fingerprint scan during vault setup should not be interpreted as a normal unlock unless the controller is actually in the unlock state.

The same idea is used for storage operations. The system distinguishes between a locked vault, an unlocked and mounted vault, an open mapper without a mount, a busy mount point, and a device that has disappeared. These states are not all the same from a security or recovery perspective. By making these states explicit, the application can choose safer recovery behavior instead of relying only on a simple locked/unlocked flag.

3.10. Design Alternatives

Before settling on the current LUKS2/dm-crypt based design, a FUSE-based filesystem approach was considered. The idea was to implement a user-space filesystem where project code would run before file operations. In that design, read operations could trigger decryption, write operations could trigger encryption, and fingerprint verification could be requested when the key was not available.

This approach was attractive because it gave very fine control over file access. The system could decide what to do for each filesystem operation and could theoretically combine authentication, encryption, and file access policy in one place. It also matched the first intuition of the project: every file operation should pass through the biometric vault logic.

However, a filesystem has many more responsibilities than simply reading and writing file contents. It must handle metadata, permissions, renaming, directory traversal, partial writes, random access, file handles, caching behavior, concurrent access, and error cases. In an early experiment, getting large-file reads and concurrent access to behave correctly already took more effort than the rest of the vault logic put together, and that was before any security feature was added on top. Large files also raised performance concerns, since every operation crosses the boundary between the kernel and the user-space filesystem process.

The design therefore moved to block-level encryption with LUKS2/dm-crypt. Encryption now sits below the filesystem layer: the Linux storage stack handles normal filesystem behavior, while the project focuses on vault setup, biometric authentication, key access, GUI workflow, and runtime safety. Put differently, the work shifted away from implementing a custom encrypted filesystem and toward controlling access to a standard encrypted storage volume.

Table 3.3: Comparison of considered storage design alternatives.

Approach	Advantages	Reasons for not choosing as main design
FUSE-based encrypted filesystem	Fine-grained control over file operations and authentication points	Complete filesystem behavior is complex; large-file performance and edge cases require careful handling
Application-level encrypted container	Simple to keep encryption inside one application	Files are mainly accessible through the application instead of behaving like normal mounted storage
LUKS2/dm-crypt encrypted block device	Standard Linux storage encryption, normal filesystem behavior after unlock, mature tooling	Requires careful key handling and mount/unmount cleanup

3.11. Security Considerations

The previous sections described the mechanics of the system. From a security perspective, the main point is the separation of trust boundaries: biometric matching, key-release control, host workflow coordination, and encrypted storage are handled by different parts of the system. This prevents the GUI or storage manager from becoming responsible for every sensitive decision.

Key exposure is intentionally limited. The host receives key material only for operations that require it, such as vault setup or unlock, and does not keep it persistently. The protected firmware-to-host channel and short host-side key lifetime reduce the situations in which key material is exposed.

Biometric data is handled with the same boundary-based approach. Fingerprint verification is treated as an authorization step rather than as a way to derive the encryption key, keeping biometric handling separate from disk encryption.

The access model is designed to be stable after setup. Authorized fingerprints are selected during vault creation, and the normal workflow does not modify the access list later. This makes vault ownership easier to reason about and avoids changing access rights after the vault has already been created.

Runtime protection is still necessary because an unlocked vault behaves like a normal mounted filesystem. Auto-lock and emergency lock reduce the time and situations in which the vault remains mounted unintentionally. Events such as sensor disconnection, abnormal runtime state, or removal of the block device are treated as security-relevant failures, not only as hardware errors.

The main security limitation is that the project protects data at rest and controls the unlock process; it does not make an already compromised host trustworthy. If the host is compromised while the vault is unlocked, mounted files may still be accessible through the operating system. For this reason, the design focuses on controlled unlock, short host-side key lifetime, consistent locking behavior, and encrypted storage when the vault is closed.

In summary, the design does not rely on a single protection point. Biometric matching limits who can request access, firmware-side key control limits where long-term key authority is placed, protected transfer and short host-side key lifetime reduce key exposure during unlock, and LUKS2/dm-crypt protects the stored data while the vault is closed. Runtime mechanisms such as auto-lock, emergency lock, and recovery handling cover the period after the vault has been opened, when ordinary applications can access the mounted filesystem.

Table 3.4: Threat and mitigation summary.

Threat or risk	Mitigation in the design	Points to consider
Lost or stolen storage device	Vault contents are protected with LUKS2/dm-crypt and AES-256-XTS	The protection applies when the vault is locked
Wrong or unauthorized fingerprint	The firmware rejects the unlock request if the fingerprint does not match	Access depends on the enrolled fingerprints and the sensor result
Biometric template exposure	Fingerprint templates remain inside the AS608 sensor	Physical attacks on the sensor are not handled by the host software
Key leakage from host storage	Host does not persistently store the vault key and clears key buffers after use	The host must still be trusted while the key is being used
Plaintext key transmission	Firmware-to-host key transfer is protected	The embedded side must also be kept secure
Vault left mounted unintentionally	Auto-lock, emergency lock, and shutdown cleanup reduce exposure time	Files are still accessible while the vault is mounted
USB storage removal during use	Device removal is treated as abnormal runtime state and triggers recovery behavior	Active file operations can still be interrupted
Busy mount during lock	Force-lock path handles processes that keep files open	The user may need to confirm closing active processes

4. IMPLEMENTATION

This chapter explains how the design described in Chapter 3 was implemented in the project. The purpose of this chapter is not to document every source file or every function, but to show how the main architectural decisions were turned into a working system.

4.1. Implementation Overview

The implementation is divided into four main parts: the Qt/QML graphical application, the host-side controller, the firmware communication layer, and the vault manager. The QML layer presents the user workflows. The `VaultController` class coordinates state and operations. The `SerialBridge` class communicates with the STM32 firmware over USB-UART. The `vault_manager` program performs system-level storage operations.

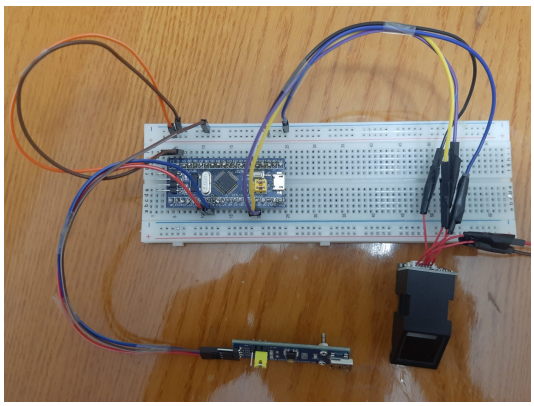
Table 4.1: Main implementation units.

Unit	Main files	Role in the project
GUI layer	QML screens	Presents vault setup, detail, lock/unlock, logs, and settings workflows
Host controller	`vault_controller.cpp`	Coordinates state, validation, registry, logs, and asynchronous operations
Serial bridge	`serial_bridge.cpp`	Sends firmware commands and receives fingerprint/key responses
Vault manager	`vault_manager.c`	Creates encrypted containers and performs mount, unmount, loop-device, and cleanup operations

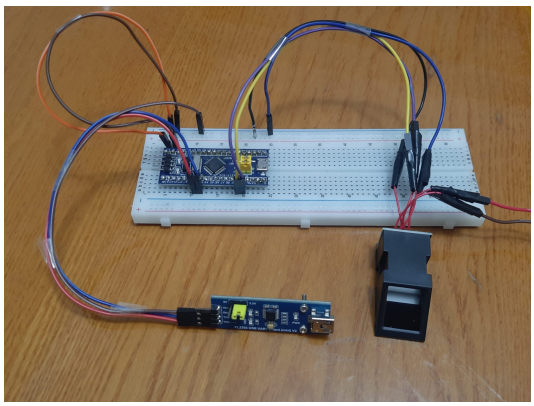
With this split, the user interface stays simple and disk operations are not placed directly inside QML. The GUI asks for an operation, the controller checks the current state, and the vault manager performs the storage action when needed.

4.2. Physical Prototype

The hardware prototype connects the AS608 fingerprint sensor to the STM32 Blue Pill, and the STM32 communicates with the host computer through the PL2303 USB-UART bridge. The physical setup was used during firmware communication tests, fingerprint enrollment and verification tests, and complete vault workflow tests.



Prototype view 1



Prototype view 2

Figure 4.1: Physical prototype including the AS608 fingerprint sensor, STM32 Blue Pill, and PL2303 USB-UART bridge.

4.3. Host Application

The host application creates a single controller instance `vaultController`. As a result, GUI can request high-level actions such as creating a vault, unlocking, locking, force-locking, creating a virtual container, or deleting a vault without knowing the low-level command sequence.

The controller stores the current runtime state of the application. Examples include the selected device, lock status, setup progress, sensor status, registered vault list, security logs, and fingerprint workflow state. These values are exposed as Qt properties, so the interface can update automatically when the state changes.

Long operations are handled asynchronously. Formatting a device, opening a vault, closing a mapper, creating a loop device, or deleting a vault may take time and may fail. For this reason, the controller uses busy flags, completion signals, and recovery paths instead of assuming that every operation finishes immediately.

4.4. Firmware Communication

The host communicates with the STM32 firmware through the `SerialBridge` class. This bridge opens the selected serial port, configures it at 115200 baud, sends compact binary packets, and parses responses from the firmware. The supported command types include enrollment, verification, deletion, cancellation, key request, and enrolled-fingerprint listing.

The serial layer allows only one fingerprint operation to be active at a time. Enrollment, verification, and key request all depend on sensor state and user interaction, so overlapping them would let the controller receive a response that belongs to the wrong workflow.

Firmware responses are converted into higher-level Qt signals such as verification completed, enrollment completed, key received, operation rejected, or connection lost. The controller then decides how each result should affect the active workflow.

4.5. Vault Manager

The `vault_manager` program is a separate command-line component because storage operations require privileges. It prepares the encrypted vault container, opens the decrypted device mapping, creates the ext4 filesystem, mounts and unmounts the vault, creates loop devices, detaches virtual containers, and deletes vault metadata when needed.

On Linux, LUKS containers are commonly managed through the `cryptsetup` tool. In this project, the vault manager starts `cryptsetup` as a child process and sends the key through a pipe connected to its standard input. The key is therefore not placed in the command-line arguments, which avoids exposing it in the process list. After the operation finishes, the key buffer is wiped from memory. Listing 4.1 shows a simplified version of this handling.

```
int pipefd[2];
pipe(pipefd);

pid_t pid = fork();
if (pid == 0) {                 /* child: replace stdin with the
    pipe */
    dup2(pipefd[0], STDIN_FILENO);
    close(pipefd[1]);
    execlp("cryptsetup", "cryptsetup", "luksOpen",
           device, mapper_name, "--key-file=-", (char *)NULL);
    _exit(127);                 /* reached only if exec fails */
```

```
}

/* parent: write the 32-byte key, then wipe it */
close(pipefd[0]);
write(pipefd[1], key, key_len);
close(pipefd[1]);
explicit_bzero(key, key_len);   /* clear key buffer after use */
```

Listing 4.1: Simplified key handling: the vault key is piped to `cryptsetup` through standard input and the buffer is cleared after use.

The same pattern is reused for vault setup, where `cryptsetup luksFormat` receives the key on standard input instead of as an argument. Passing `-key-file=-` keeps the secret out of both the argument list and any shell history.

The vault manager also includes cleanup-oriented commands. Normal locking unmounts the filesystem and closes the mapper. Force-lock handles busy mount points by verifying that the mount belongs to the vault before stopping holder processes. Reconciliation is used to clean up leftover mapper or mount state after abnormal situations.

4.6. Runtime Recovery

Several parts of the implementation are written around the possibility of partial failure. A vault may be formatted but not mounted, a mapper may remain open after an interrupted setup, or a mounted filesystem may become busy. An early version of the prototype made this concrete: when a mount failed, the dm-crypt mapper was sometimes left open, and the next unlock attempt then failed with a “device already exists” error until the mapper was closed by hand. That bug is the reason the controller and vault manager now distinguish between clean, mapper-only, mounted, busy, and missing-device states instead of a single locked/unlocked flag.

Emergency closing is also connected to application shutdown and abnormal events. If the application exits while the vault is open, or if the runtime state becomes unsafe, the controller attempts to close the vault through the vault manager. This implementation detail supports the design goal of avoiding unstable intermediate states.

Overall, the implementation follows the design principle of keeping responsibilities separate: QML handles interaction, the controller handles workflow state, the serial bridge handles firmware communication, and the vault manager handles storage operations.

5. TESTING AND EVALUATION

This chapter describes how the digital vault system was tested and evaluated. Functional workflows, firmware communication, software components, and several failure cases were tested to check whether the system remains understandable and recoverable during abnormal situations.

5.1. Testing Approach

The project was tested with a combination of manual functional testing, unit-level software tests, communication tests, filesystem usage tests, and stress tests. Manual tests were used for complete user workflows such as creating a vault, enrolling a fingerprint, unlocking the vault, locking it again, and deleting it. These tests were important because the project includes hardware, operating-system storage commands, and GUI state transitions that need to work together.

Unit-level tests were used for parts of the software that could be tested without the complete hardware setup. These tests focused on logic such as state handling, storage planning, validation behavior, and expected responses to different operation results.

Serial communication was tested before the graphical interface was fully connected to the firmware flow. External test programs were used to check command and response behavior independently from the GUI. This made it easier to verify the communication protocol before combining it with the user interface and vault workflows.

In the following tables, the status column summarizes the result of each test. *Passed* means that the expected behavior was observed during testing. If a test only satisfies the expected behavior with an important limitation, it can be marked as *Partially passed*; however, the listed tests in this chapter were completed successfully. The tests were not treated as formal performance benchmarks. This is because the project does not implement a new filesystem or a custom file-operation layer. After the vault is unlocked and mounted, regular file operations such as reading, writing, copying, moving, and deleting are handled by the operating system and the standard filesystem inside the encrypted container. Therefore, the main purpose of the tests was to check that the vault reached the expected final state after normal use, repeated use, and interrupted workflows.

Table 5.1: Test strategy overview.

Test type	Purpose	Example checks
Functional workflow tests	Verify complete user-facing vault operations	Create vault, enroll fingerprint, format, unlock, lock, delete
Unit-level software tests	Verify isolated software behavior without the full hardware setup	State transitions, validation checks, operation result handling
Serial communication tests	Verify firmware command and response behavior before GUI integration	Enroll, verify, cancel, timeout, failure responses
Filesystem usage tests	Verify normal file operations after the vault is unlocked	Read, write, copy, move, delete files
Stress tests	Verify behavior under repeated or heavier usage	Repeated lock/unlock cycles, larger file operations
Failure and edge case tests	Verify recovery behavior when hardware or storage state changes unexpectedly	Wrong fingerprint, USB removal, STM32 disconnection, killed process

5.2. Functional Test Cases

The main functional tests checked whether the system can complete the expected vault lifecycle operations. These tests were performed from the user-facing workflow, so they verified not only the storage operation itself but also the GUI state transitions around the operation.

Table 5.2: Functional test cases.

Test case	Expected behavior	Observed result	Status
Vault creation	User selects a storage target and starts setup	Setup flow started correctly	Passed
Fingerprint enrollment	User enrolls a fingerprint for vault access	Fingerprint slot was registered	Passed
Vault formatting	Selected storage is prepared as an encrypted vault	Encrypted container and filesystem were created	Passed
Vault unlock	Authorized fingerprint opens the vault	Vault became accessible at the mount point	Passed
Vault lock	Mounted vault is closed from the GUI	Filesystem was unmounted and mapper was closed	Passed
Force-lock	Busy vault mount is force-closed after confirmation	Holder processes were handled and the vault was locked	Passed
Virtual container creation	User creates a virtual storage file as a vault target	Virtual container was attached as a block device	Passed
Vault deletion	User deletes a managed vault through the workflow	Managed vault state was removed from the application	Passed

5.3. Failure and Edge Case Tests

Failure tests were important because the project can interact with removable devices and external hardware. A vault application should not only work when every component behaves correctly; it should also respond clearly when a component disappears or an operation is interrupted.

Table 5.3: Failure and edge case test cases.

Test case	Expected behavior	Observed result	Status
Wrong fingerprint	Unlock is rejected and vault remains locked	Verification failed and vault did not open	Passed
USB storage removal	Runtime state is reconciled after device removal	Removal was detected as an abnormal state	Passed
STM32 disconnection	GUI reports sensor loss and stops fingerprint flow	Disconnection was reflected in the interface	Passed
Application process killed	Remaining mapper or mount state is cleaned during recovery	Reconciliation and emergency close behavior were used	Passed
Busy mount during lock	Busy state is reported instead of silently failing	Force-lock path handled the busy mount	Passed
Interrupted operation	GUI avoids showing a misleading successful state	Controller state was updated after failure or cancellation	Passed

These tests also helped verify the importance of the state-machine style design described in Chapter 3. Pulling the USB cable of the STM32 in the middle of an unlock was one of the more useful cases: the first builds simply kept waiting for a response that never came, and only after the controller learned to treat a dropped connection as a failure did the interface recover cleanly. A fingerprint result, likewise, should only affect the workflow that is currently active, and a storage failure should not leave the interface showing a successful state.

5.4. Filesystem Usage and Stress Tests

After the vault was unlocked and mounted, normal filesystem behavior was also tested. This was important because the user should not need to think about the encryption layer during everyday file usage. The opened vault should behave like a normal folder while it is unlocked.

Table 5.4: Filesystem usage and stress test cases.

Test case	Expected behavior	Observed result	Status
File write	User writes a file into the unlocked vault	File was created successfully	Passed
File read	User opens a stored file from the vault	File content was readable after unlock	Passed
File copy	User copies files into and inside the vault	Copy operations completed normally	Passed
File move	User moves or renames files in the vault	Move and rename operations completed normally	Passed
File delete	User deletes files from the unlocked vault	Deleted files were removed from the vault	Passed
Large file operation	User copies or writes a larger file into the vault	Operation completed without breaking vault state	Passed
Repeated lock/unlock	User repeatedly opens and closes the vault	Vault returned to the expected state after each cycle	Passed

These tests were also a practical check for the storage design decision. Since the project uses block-device encryption, the filesystem inside the vault remains a standard filesystem after unlock. Therefore, normal file operations are handled by the operating system rather than by custom file-operation code. In the filesystem tests, the important observation was not only that individual file operations succeeded, but also that the vault could still be locked afterwards without leaving an open mapper or stale mount point behind.

5.5. Serial Communication Tests

The serial communication layer was tested before relying on the GUI workflow. The goal was to confirm that commands such as enroll, verify, cancel, delete, list enrolled fingerprints, and key request could be sent and interpreted correctly. Testing this layer separately made debugging easier because communication problems could be identified before they appeared as GUI workflow problems.

The communication tests also checked timeout, busy, failed verification, and sensor-error responses. These cases are important because fingerprint operations depend on user action and sensor state. A delayed or failed response should not block the whole application indefinitely.

Table 5.5: Serial communication test cases.

Test case	Expected behavior	Observed result	Status
Enroll command	Firmware starts enrollment and returns a clear result	Enrollment response was received and interpreted	Passed
Verify command	Firmware returns success only for an authorized fingerprint	Verification result was handled by the host flow	Passed
Cancel command	Active fingerprint operation is cancelled cleanly	Cancel response returned control to the application	Passed
Timeout response	Host stops waiting and shows a recoverable state	Timeout was treated as operation failure	Passed
Sensor error response	Host reports the error and stops the current flow	Error response was handled without opening the vault	Passed
Key request	Key is requested only after successful authorization	Key request was tied to the authenticated workflow	Passed

5.6. Evaluation Summary

The tests showed that the main vault workflows were functional: vault creation, fingerprint enrollment, formatting, unlocking, locking, force-locking, virtual-container usage, and deletion were exercised. Normal filesystem operations such as reading,

writing, copying, moving, and deleting files were also tested after unlock. Stress tests with repeated operations and larger file usage helped check that the vault did not remain in a misleading or unstable state. The most useful observations came from cases where the system was interrupted: a removed storage device, a disconnected STM32, or a killed application process should not leave the GUI claiming that the vault is safely open or safely closed without checking the real mapper and mount state.

The most important result of the testing phase was confirming that the system should not be treated as a simple locked/unlocked toggle. The actual runtime state may include an open mapper, a mounted filesystem, a busy mount, a missing device, or a disconnected sensor. Testing these cases helped validate the design decision to use explicit state tracking and emergency recovery paths.

The completed tests were sufficient to evaluate the main functional behavior, normal unlocked filesystem usage, stress behavior, and the most important failure-handling paths of the prototype.

6. CONCLUSION AND FUTURE WORK

This project presented a biometric based encrypted digital vault that combines fingerprint-controlled access with standard Linux storage encryption. The main goal was to make encrypted storage usable without turning the user workflow into a command-line or password-management task. For this purpose, the system was designed around three main parts: an embedded biometric access unit, a host application with a graphical interface, and a vault management layer that prepares and controls encrypted storage.

The final design uses the AS608 fingerprint sensor and STM32 Blue Pill as the biometric access side of the system, while the host application manages the user workflow and encrypted storage operations. After successful authorization, the host can unlock a LUKS2/dm-crypt based vault and present it as a normal mounted folder. This keeps the everyday storage behavior close to the user's existing mental model: the user unlocks the vault, works with files normally, and locks the vault again when finished.

The project also supports both physical storage devices and virtual storage containers. This makes the system more flexible because a user can prepare an external device as a vault or create a virtual block device from a storage file. In both cases, the same vault workflow can be used from the graphical interface.

6.1. Project Outcomes

The implemented system covers the main vault lifecycle operations: vault creation, fingerprint enrollment, formatting, unlocking, locking, force-locking, virtual container creation, and deletion. Authorized fingerprints are enrolled during vault creation, which keeps the access policy clear for the lifetime of the vault. The graphical interface hides low-level storage commands while still showing important system states such as locked/unlocked status, sensor connection, active operations, and abnormal conditions.

An important design decision was to use block-device encryption with LUKS2/dm-crypt instead of a custom FUSE-based filesystem. This choice gave up fine-grained per-operation control, but it kept the standard Linux storage stack in charge of normal filesystem behavior, as discussed in Chapter 3.

Another important result was the state-oriented design of the host application. Testing showed that the system cannot be represented only as locked or unlocked. A vault may have an open mapper, a mounted filesystem, a busy mount point, a miss-

ing storage device, or a disconnected fingerprint unit. The implementation therefore includes explicit state tracking, recovery behavior, and emergency locking paths.

6.2. Limitations

The project uses established components rather than proposing a new encryption algorithm or a new filesystem standard. This is a deliberate choice because the main contribution is the integration of biometric authorization, embedded control, graphical interaction, and encrypted storage into a usable vault workflow.

The main limitations are related to key protection boundaries and cryptographic flexibility. The vault key is controlled by the firmware side, but the hardware platform still has physical extraction risks. A stronger hardware platform, protected storage, or secure element would reduce the risk of key leakage from the embedded device.

Another limitation is that the key is still present on the host machine for a short time while the vault operation is performed. The implementation avoids persistent storage of the key and wipes the key buffer after use, but a compromised host operating system could still be a risk while the key or unlocked vault is active. This is a general limitation of systems where the host must temporarily open encrypted storage.

The current system is also centered on one storage encryption configuration. AES-XTS is appropriate for this project because it is widely used for disk encryption, but future versions could support additional encryption algorithms or configurations if the storage backend and security requirements change.

6.3. Future Work

One future improvement is strengthening the embedded side of the system. A more secure hardware platform, secure element, or stronger firmware protection could reduce the risk of physical attacks against the device. This would also make the separation between biometric authorization and host-side storage control stronger.

Another improvement is reducing host-side key exposure as much as possible. This could include stricter memory handling, shorter key lifetime on the host, process isolation, and additional checks around when the key can be requested and used.

The user interface can also be improved with more detailed recovery guidance and clearer reporting for abnormal states. The current interface already follows a state-oriented workflow, but future versions could provide more structured diagnostics when a device is removed, a mount is busy, or the fingerprint unit is disconnected.

The storage layer can also be extended with configurable encryption options. The current design uses a standard disk-encryption configuration, but supporting additional

algorithms or profiles could make the project adaptable to different requirements.

6.4. Final Remarks

Overall, the project shows that biometric access control can be combined with standard encrypted storage tools to build a usable digital vault workflow. The work does not depend on a new cryptographic primitive; instead, it focuses on connecting existing security mechanisms with a clear hardware and software architecture. The most valuable part of the work was not only opening and closing an encrypted container, but also handling the surrounding states that occur in a real system: hardware disconnection, removable storage behavior, interrupted operations, and user-facing recovery. These decisions shaped the final architecture and provide a clear direction for future improvements.

BIBLIOGRAPHY

[1] National Institute of Standards and Technology, "Advanced Encryption Standard (AES), FIPS 197," National Institute of Standards and Technology, Tech. Rep., 2023. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.197-upd1 (visited on 06/11/2026).

[2] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC Editor, Tech. Rep. RFC 8439, 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8439 (visited on 06/14/2026).

[3] R. Anderson, E. Biham, and L. Knudsen, "Serpent: A Proposal for the Advanced Encryption Standard," University of Cambridge Computer Laboratory, Tech. Rep., 1998. [Online]. Available: https://www.cl.cam.ac.uk/~rja14/Papers/serpent.pdf (visited on 06/14/2026).

[4] B. Schneier, J. Kelsey, D. Whiting, D. Wagner, C. Hall, and N. Ferguson, "Twofish: A 128-Bit Block Cipher," Counterpane Systems, Tech. Rep., 1998. [Online]. Available: https://www.schneier.com/wp-content/uploads/2016/02/paper-twofish-paper.pdf (visited on 06/14/2026).

[5] M. Matsui, J. Nakajima, and S. Moriai, "A Description of the Camellia Encryption Algorithm," RFC Editor, Tech. Rep. RFC 3713, 2004. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3713 (visited on 06/14/2026).

[6] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Storage Devices," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-38E, 2010. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-38E (visited on 06/11/2026).

[7] cryptsetup Project. "Cryptsetup documentation." (2026), [Online]. Available: https://gitlab.com/cryptsetup/cryptsetup/-/wikis/home (visited on 06/11/2026).

[8] The Linux Kernel Documentation. "Device Mapper Crypt Target (dm-crypt)." (2026), [Online]. Available: https://docs.kernel.org/admin-guide/device-mapper/dm-crypt.html (visited on 06/11/2026).

[9] The Linux Kernel Documentation. "ext4 Filesystem." (2026), [Online]. Available: https://docs.kernel.org/filesystems/ext4/index.html (visited on 06/11/2026).

[10] Microsoft. “Bitlocker overview.” (2026), [Online]. Available: https://learn.microsoft.com/en-us/windows/security/operating-system-security/data-protection/bitlocker/ (visited on 06/11/2026).

[11] S. Skorobogatov, “Teardown and feasibility study of ironkey: The most secure usb flash drive,” *arXiv preprint arXiv:2110.14090*, 2021. [Online]. Available: https://arxiv.org/abs/2110.14090 (visited on 06/11/2026).

[12] P. Swierczynski, M. Fyrbiak, P. Koppe, A. Moradi, and C. Paar, “Interdiction in practice: Hardware trojan against a high-security usb flash drive,” *arXiv preprint arXiv:1910.00947*, 2019. [Online]. Available: https://arxiv.org/abs/1910.00947 (visited on 06/11/2026).

[13] libfuse Project. “Libfuse: The reference implementation of the linux fuse interface.” (2026), [Online]. Available: https://github.com/libfuse/libfuse (visited on 06/11/2026).

[14] The Linux Kernel Documentation. “FUSE (Filesystem in Userspace) Technical Documentation.” (2026), [Online]. Available: https://docs.kernel.org/filesystems/fuse/index.html (visited on 06/11/2026).

APPENDICES

Appendix A: Serial Communication Protocol Summary

The host application communicates with the STM32 firmware over a compact binary UART protocol. The purpose of this appendix is to summarize the protocol surface used by the project without moving low-level implementation details into the main chapters.

Each packet starts with a synchronization byte, followed by a command or response code, payload length, optional payload bytes, and a checksum byte. The checksum is calculated as the XOR of the preceding packet bytes. The maximum payload size is 64 bytes, and the vault key payload is 32 bytes.

Table 6.1: UART packet format summary.

Field	Size	Description
Sync	1 byte	Fixed packet start marker, `0xAA`
Code	1 byte	Command, event, or response code
Length	1 byte	Number of payload bytes
Payload	0–64 bytes	Command or response data, depending on the packet code
Checksum	1 byte	XOR checksum over sync, code, length, and payload bytes

Table 6.2: Host-to-firmware command summary.

Command	Code	Purpose
Enroll start	`0x01`	Starts fingerprint enrollment for the requested slot or automatic slot selection
Verify start	`0x02`	Starts fingerprint verification and returns the matched authorized fingerprint if successful
Delete	`0x03`	Deletes an enrolled fingerprint template from the sensor slot
Cancel	`0x04`	Cancels an active fingerprint operation
Get key	`0x05`	Requests vault key material after successful verification
List enrolled	`0x06`	Lists enrolled fingerprint slots known by the firmware/sensor flow
Status	`0x07`	Checks firmware/sensor availability and status

Table 6.3: Firmware event and response summary.

Code group	Codes	Meaning
User interaction events	`0x10-0x14`	Waiting for finger, finger detected, remove finger, place again, or processing
Successful result	`0x20`	Operation completed successfully
Expected failure results	`0x21-0x28`	No finger, bad image, no match, already exists, slot full, invalid ID, cancelled, or timeout
Key data	`0x29`	Returns 32 bytes of key material after authorization
Busy or hardware error	`0x2A-0x2B`	Firmware is busy or the sensor reported an error